



Parallélisme à base de thread

(PABT 2025-2026)

TD1

Programmation par tâches OpenMP

teodora.hovi@cea.fr

Ce TD est noté. Un rapport au format PDF comprenant les réponses aux questions textuelles ainsi que des explications détaillées sur le code est attendu (voir lien en fin d'énoncé). **Une réponse constituée du code seul ne se verra attribuer qu'une fraction des points.**

L'utilisation de chaque directive OpenMP et chaque clause associée devra ainsi trouver justification dans le rapport.

Rappels

- Pour compiler un programme C utilisant OpenMP avec le compilateur gcc :
`% gcc -fopenmp source.c -o exec`
- La variable d'environnement `OMP_NUM_THREADS` permet de spécifier le nombre de threads OpenMP :
Option 1 : `% export OMP_NUM_THREADS=4 ; ./monprog`
Option 2 : `% OMP_NUM_THREADS=4 ./monprog`

I Passage de OpenMP à Pthread

Q.1: Étudier le programme `omp_omp2pth/prog_omp.c`. Quelle doit être la valeur de la variable `sum` à la fin de l'exécution ? Compiler et exécuter le programme avec 4 threads.

Q.2: Réécrire ce programme avec les threads POSIX.

Q.3: Revenir sur le programme OpenMP. La boucle `for` est-elle parallélisée ? Pourquoi ? Modifier le programme pour que la boucle `for` soit distribuée sur tous les threads OpenMP. Quelle est la valeur attendue de la variable `sum` ?

Q.4: Quelle directive assure que plusieurs threads ne modifient pas `sum` simultanément ? Est-ce la directive la plus appropriée ? Pourquoi ?

Q.5: OpenMP propose-t'il un autre moyen, plus simple et plus efficace, pour calculer `sum` ? Si oui, effectuer la modification.

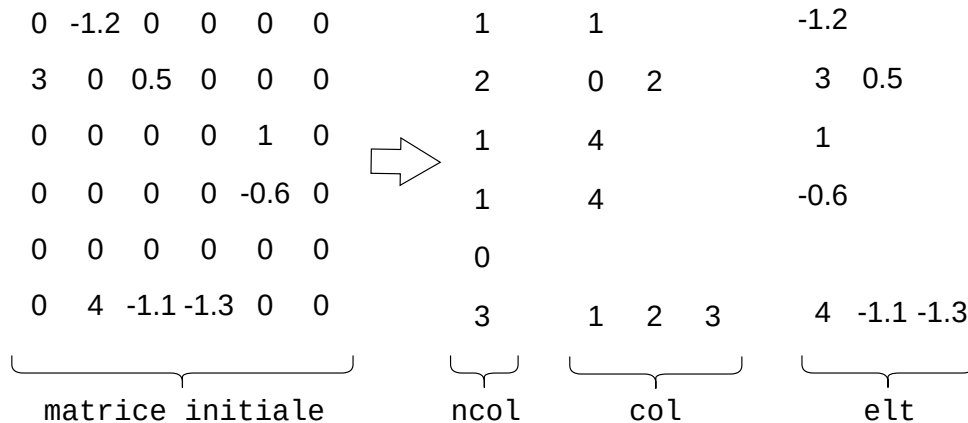
Q.6: Reprendre le calcul parallèle du max d'un tableau implémenté durant le TD1 de Pthread (**Q.7** et **Q.8**). En écrire une version OpenMP.

II Produit matrice-creuse vecteur

Une matrice creuse est une matrice dont "la plupart" des éléments sont nuls. Ici, la matrice a au plus cinq éléments non nuls par ligne.

Afin de minimiser la taille de la structure en mémoire, on ne stocke que les éléments non nuls. La structure `sparse_matrix_t` contient donc, pour chaque ligne i :

- le nombre d'éléments non nuls (*i.e.* le nombre de colonnes "utiles") : `ncol[i]`
- l'indice des colonnes "utiles" : `col[i][k]` pour $0 \leq k < \text{ncol}[i]$
- la valeur des éléments non nuls : `elt[i][k]` pour $0 \leq k < \text{ncol}[i]$



On veut paralléliser le produit d'une matrice creuse par un vecteur. Pour mesurer le coût de l'opération, on effectue ce produit un grand nombre de fois :

```
./exec <N> <niter>
```

où N est la taille de la matrice

et `niter` le nombre de répétitions de l'opération

Le fichier `omp_prod_mat_vec/prod_mat_vec_seq.c` définit la structure et l'initialisation aléatoire d'une matrice creuse et d'un vecteur.

Q.7: La fonction `prod_mat_vec` implémente la multiplication séquentielle d'une matrice creuse par un vecteur. Implémenter `prod_mat_vec_par`, la version parallélisée avec OpenMP de ce même calcul.

Q.8: Paralléliser la fonction `is_equal`.

Q.9: Relever les temps d'exécution pour 1, 2, 4 et 8 threads OpenMP avec les paramètres suivants :

`N = 1000000`

`niter = 100`

Commenter.

TD à rendre sur ce lien (préciser Prénom NOM) avant 18h le 22/01/2026, à 23h59.

¹ <https://e.pcloud.com/#page=puplink&code=GLL7ZRGGr7gi41eKBy2rClC2uTDL7VYOKy>